

Probabilistic reasoning



Graphical models

- A graphical notation for representing conditional independence assertions,
 - compact specification of full joint distributions.
- Graph-based syntax:
 - It is composed by a set of nodes, one per variable.
 - A directed, acyclic graph (link $\approx$ "directly influences").
 - A conditional distribution for each node given its parents:
 $\mathbf{P}(X_i \mid \text{Parents}(X_i))$
- In the simplest case, conditional distribution represented as a **conditional probability table** (CPT) giving the distribution over X_i for each combination of parent values.

The syntax

- A graph G consists of
 - a set of nodes-vertices $V = \{V_1, V_2, \dots, V_n\}$
 - A set of edges $E = \{E_1, E_2, \dots, E_n\}$.
- An edge that connects a pair of nodes V_i, V_j can be of two types:
 - Directed (represented by $V_i \rightarrow V_j$) and undirected (represented by $V_i - \underline{V_j}$).
- A graph can also be represented as an adjacency matrix,
 - Undirected graph: if the position $G(i,j)$ contains 1, indicates an edge between i and j vertices.
 - Directed graph, a value of 1 or -1 indicates the direction of the edge.

The syntax

- Bayesian networks – directed, acyclic graphs (short: DAGs)
- Markov networks – undirected graphs

The chain rule

(chain rule of probability)

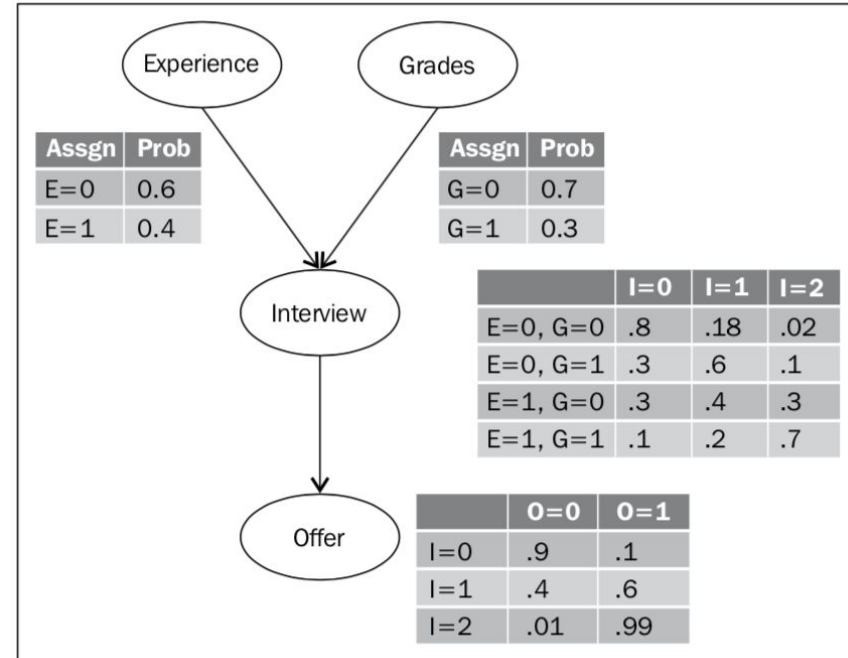
$$P(E, G, I, O) = P(E) \times P(G|E) \times P(I|E, G) \times P(O|E, G, I)$$

(chain rule for Bayesian networks)

$$P(E, G, I, O) = P(E) \times P(G) \times P(I|E, G) \times P(O|I)$$

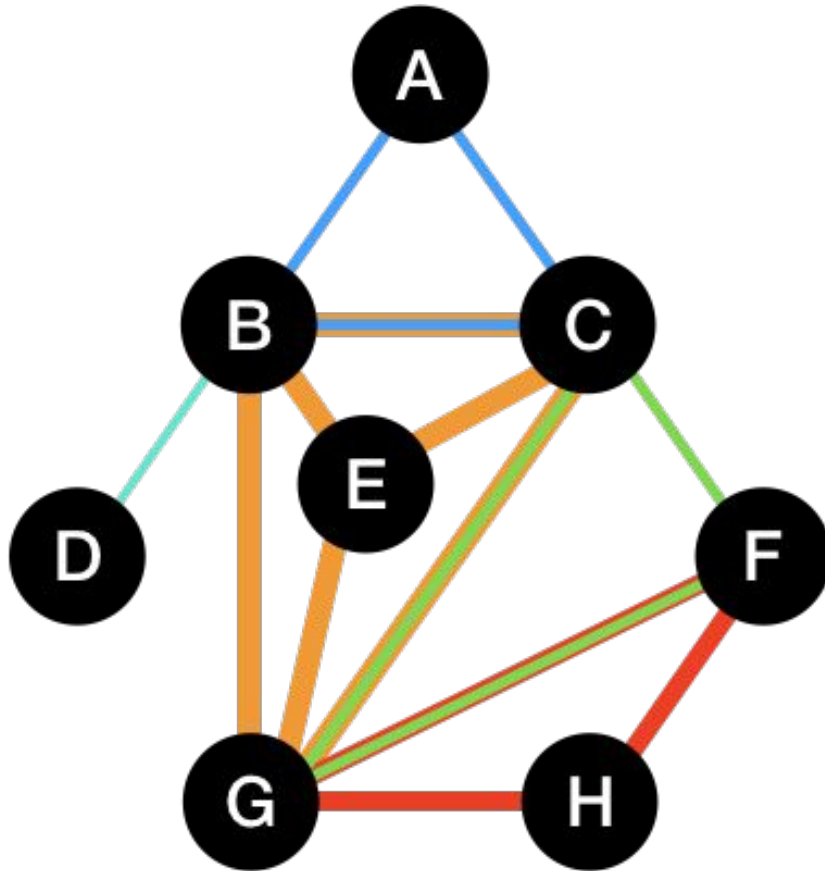
More generally, it can be written as:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$



- Use probability tables to represent their relationships.
- Dependence structure is characterized by a graph.
- Inference □ posterior distribution.

In a Markov network



The probability of an attribute is conditioned to the other attributes that are within the same clique.

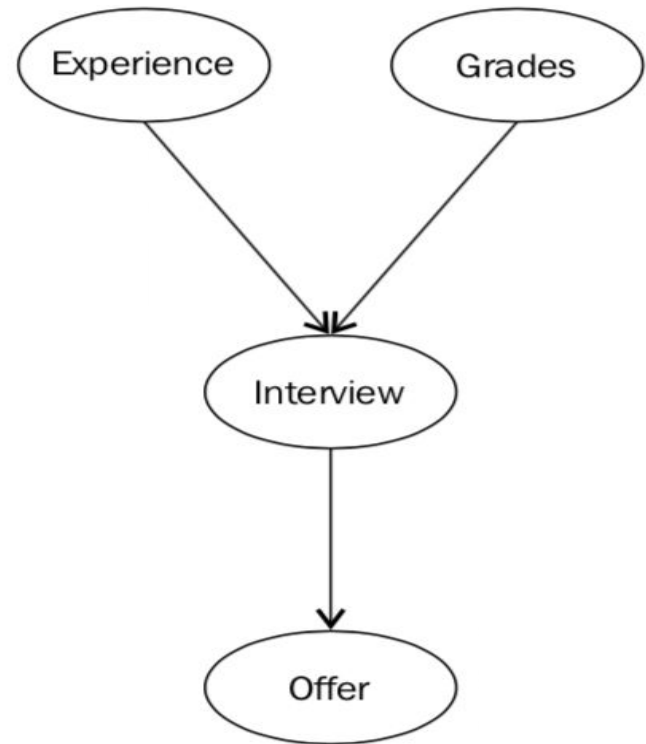
Compactness in defining the joint distribution

- A full joint distribution of n attributes needs is $O(2^n)$.
- A CPT for Boolean X_i with k Boolean parents has 2^k rows for the combinations of parent values.
 - If each variable has no more than k parents, the complete network requires $O(n \cdot 2^k)$ numbers.



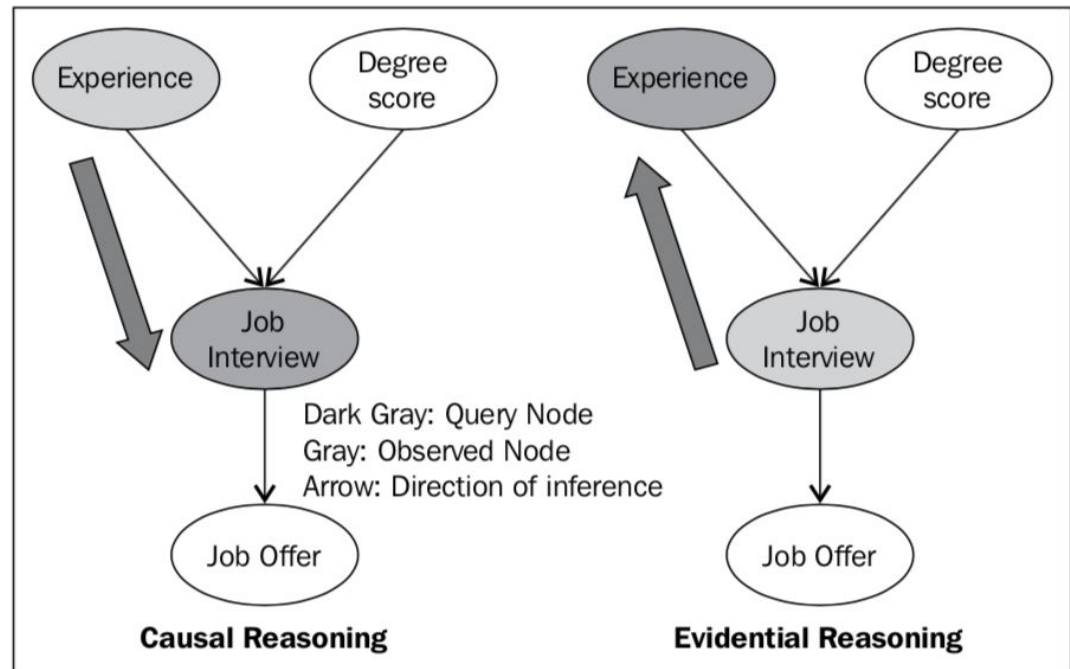
Example

- A candidate with a certain amount of experience and education is looking for a job.
- The candidate is also applying for admission to a higher education program.
- We are trying to fully specify the joint distribution over the job offer.



Reasoning patterns

- Causal
- Evidential
- Intercausal

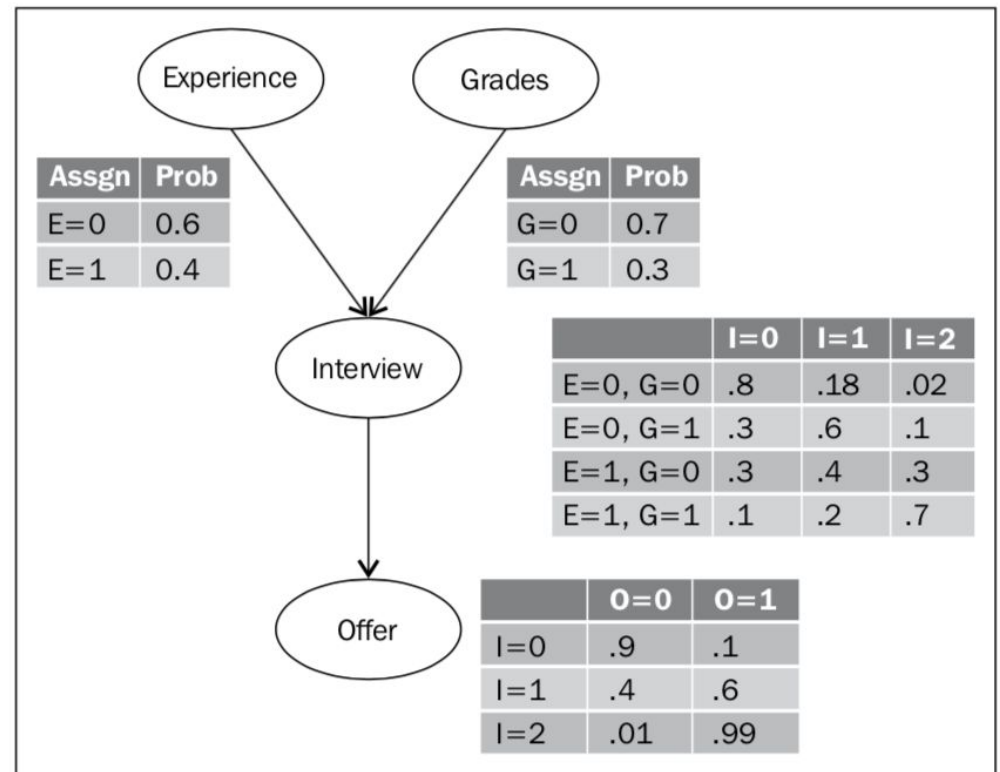


Causal reasoning

$$P(O=1) = 0.43$$

$$P(O=1|G=0) = 0.35$$

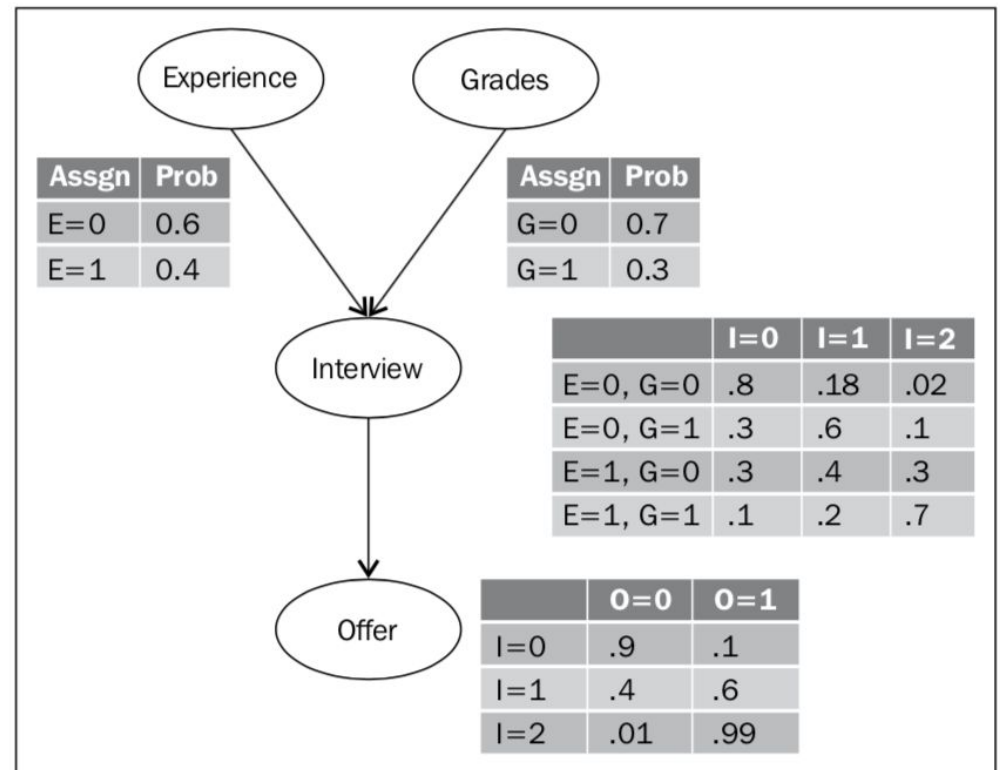
$$P(O=1|G=0, E=0) = 0.21$$



Evidential reasoning

$$P(E=1) = 0.40$$

$$P(E=1|I=2) = 0.86$$

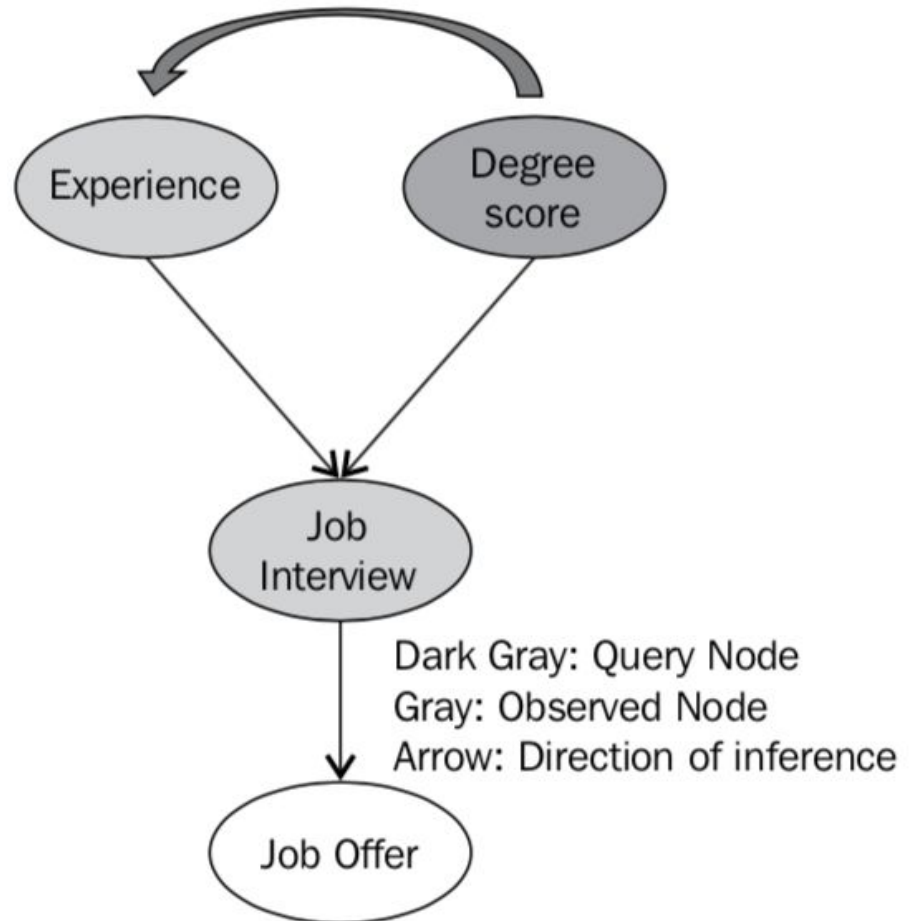


Intercausal reasoning

$$P(E=1) = 0.4$$

$$P(E=1|I=2) = 0.86$$

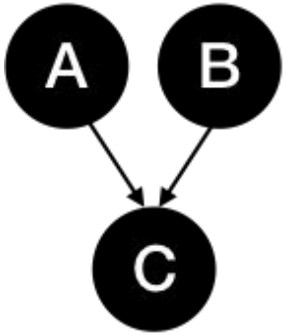
$$P(E=1|I=2, G=0) = 0.91$$



The separation concept

- The separation concept allows to understand how exactly information flows through a Bayes network.
 - Are there conditions where influence does not flow?
- The separation concept allows to build the rules that can be used to interpret a graphical model.
 - These rules are in the form of **conditional independence statements**.

Example: meal quality



A quality of ingredients

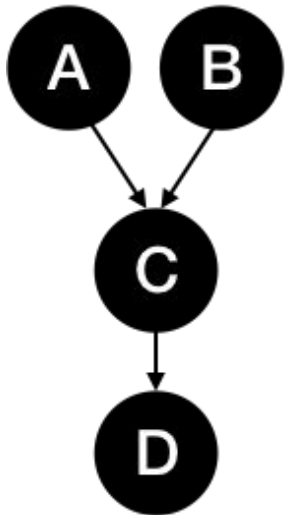
B cook's skill

C meal quality

- If C is not known, A and B are independent.
- If C is known, then A and B become (conditionally) dependent given C.

$$A \not\perp B | C$$

Example: meal quality



A quality of ingredients

B cook's skill

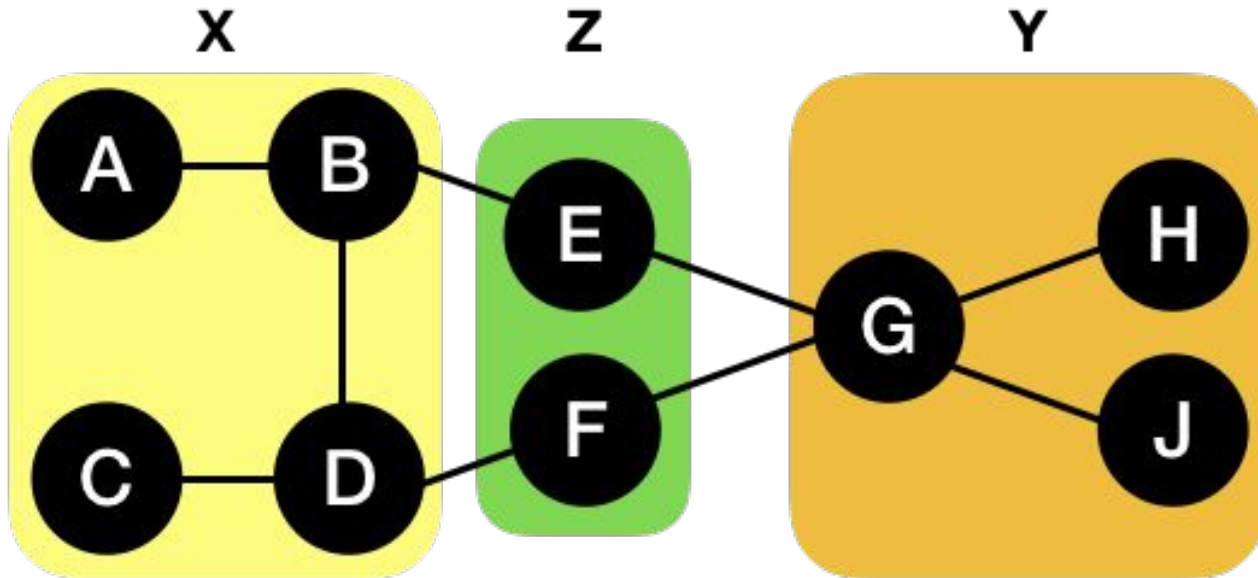
C meal quality

D restaurant success

- If nothing is known about the restaurant success or meal quality or both, the cook's skills and quality of the ingredients are unrelated, that is, independent.
- However, if we observe that the restaurant has no success, we can infer that the meal quality might be bad.
- If we further learn that the ingredients quality is high, we will conclude that the cook's skills must be low, thus rendering both variables dependent.

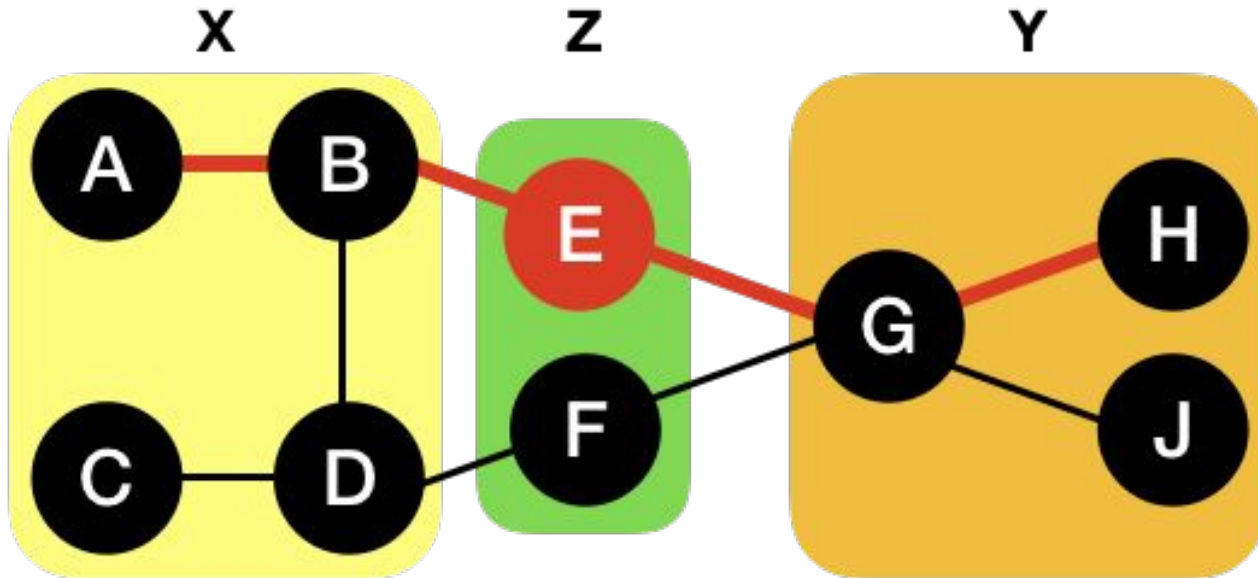
$$A \not\perp\!\!\!\perp B | D$$

u-separation



- Let X , Y , Z be three disjoint subsets of nodes.
- Separation criteria:
 - Z u-separates X from Y — written as $X \perp_G Y | Z$, if every possible path from a node in X to a node in Y is **blocked**.
 - A path is blocked if it contains one (or more) blocking nodes.
 - A node is a blocking node if it lies in Z .

u-separation



- E.g. path A–B–E–G–H is blocked by $E \in Z$.
- Every path from X to Y is blocked by Z.
- Hence,

$$A, B, C, D \perp\!\!\!\perp G, H, J \mid E, F$$

D-separation

- Separation criterion for directed graphs uses the same principles of u-separation.
- Two modifications are necessary:
 - Directed paths may lead also in reverse to the arrows.
 - The blocking node condition is more sophisticated.

D-separation

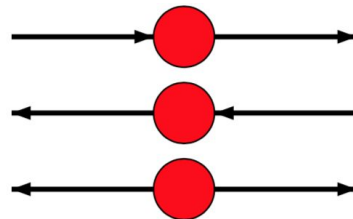
- **Blocking Node (in a directed path)**

A node A is blocking if its edge directions along the path

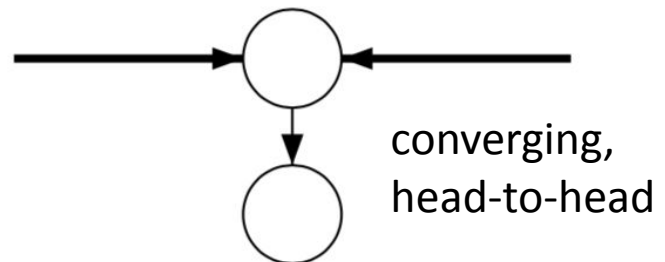
- are of type 1 and $A \in Z$, or
- are of type 2 and neither A nor one of its descendants is in Z .

Type 1

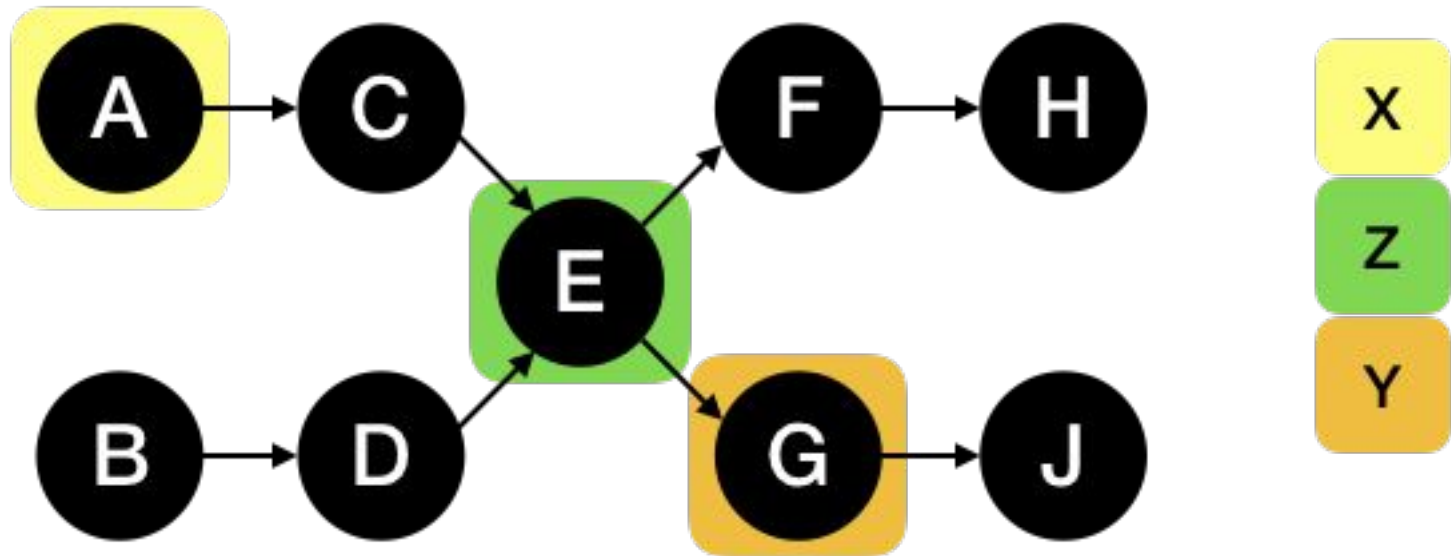
Serial, head-to-tail.
Serial, head-to-tail.
Diverging, tail-to-tail.



Type 2



D-separation



Checking path $A \rightarrow C \rightarrow E \rightarrow G$

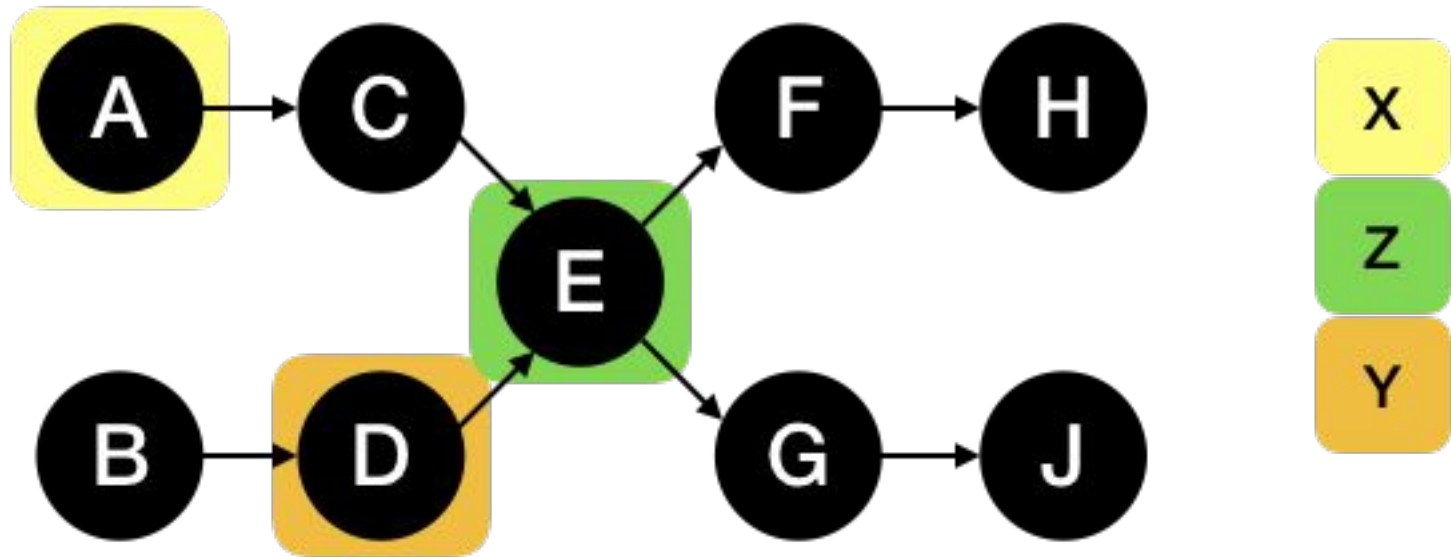
C is serial and not in Z: **non-blocking**

E is also serial but in Z: **blocking**

Path is blocked, no other paths between A and G are available

$$A \perp\!\!\!\perp G \mid E$$

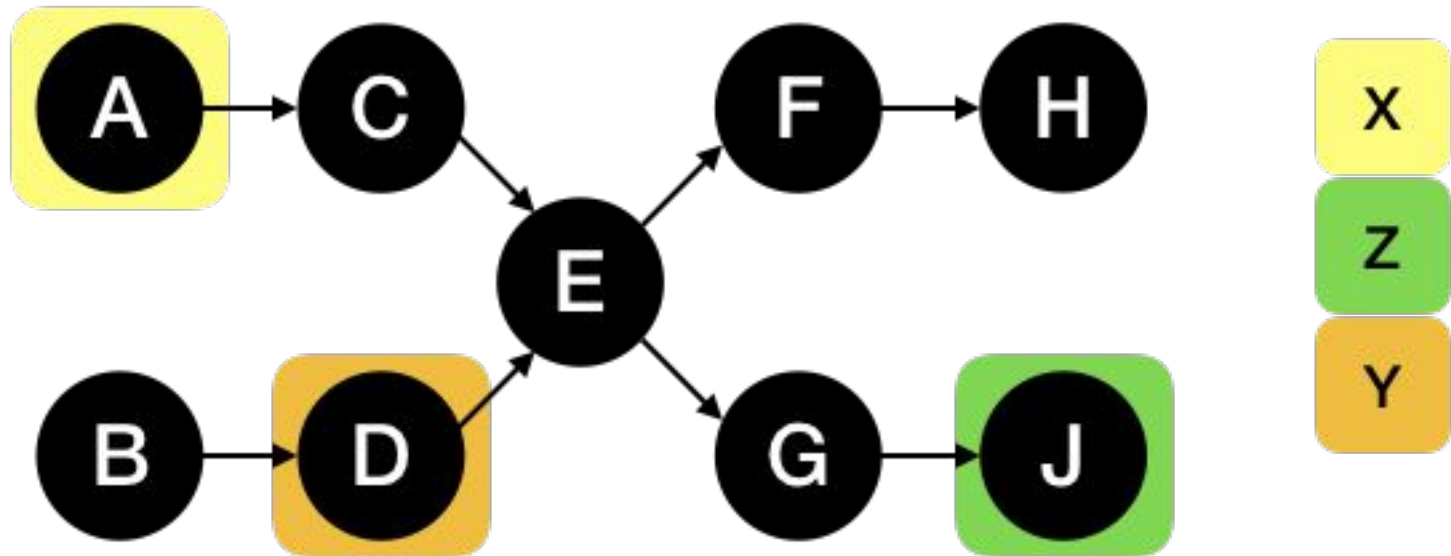
D-separation



Checking path $A \rightarrow C \rightarrow E \leftarrow D$
C is serial and not in Z: **non-blocking**
E is converging and in Z: **non-blocking**
Path is not blocked

$$A \not\perp\!\!\!\perp D | E$$

D-separation



Checking path $A \rightarrow C \rightarrow E \leftarrow D$

C is serial and not in Z: **non-blocking**

E is converging and not in Z but one of its descendants (J) is in Z: **non-blocking**

Path is not blocked

$$A \not\perp\!\!\!\perp D | J$$

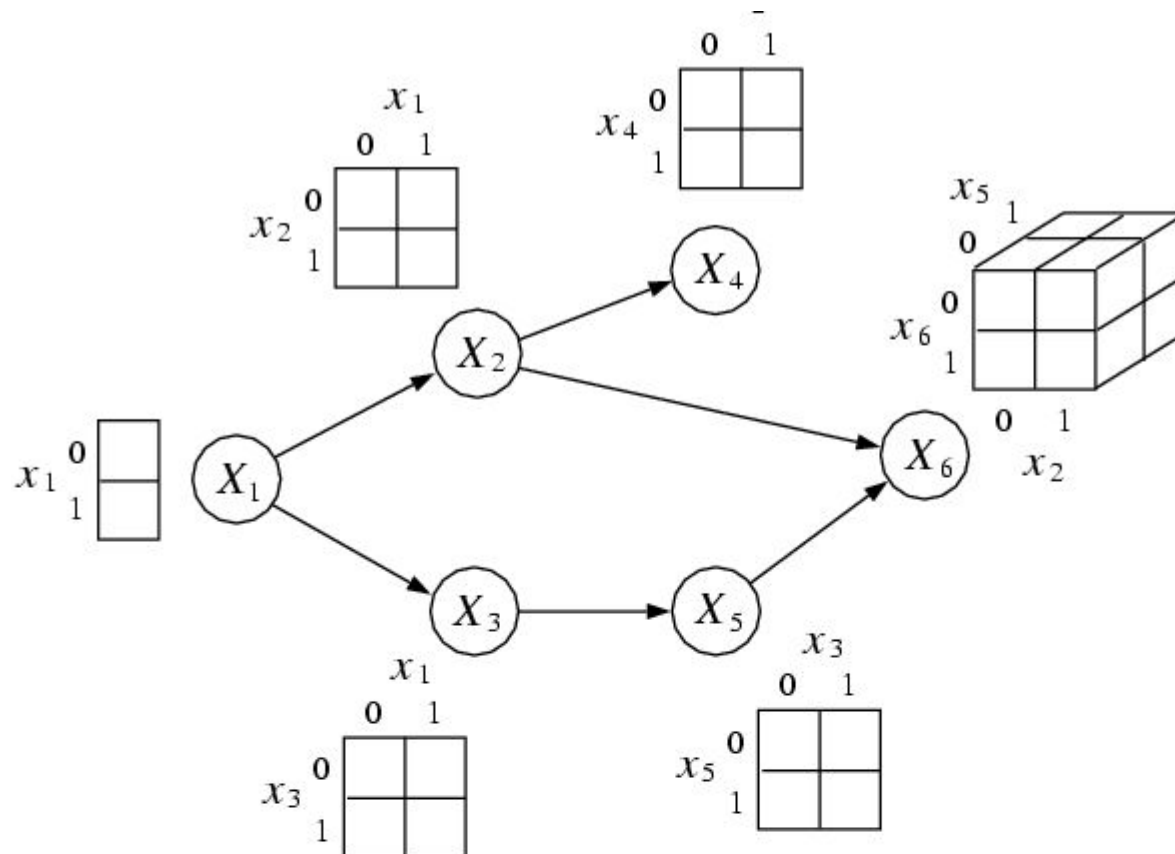
Working with Bayesian Networks

- **Parameters learning: CDF estimation**
- Structure learning
- Bayesian Inference

Bayesian Graphical Models

Parameter learning

- Conditional Probability Tables (CPTs)



MLE for Bayesian networks

- Consider the simplest Bayesian network with two binary random variables, X and Y ,

$$X \rightarrow Y$$

- This network has two CPDs that are parameterized by:
 - θ_x (for X_0 and X_1) and
 - $\theta_{y|x}$ (for the four entries in the CPD for Y given X).
- The dataset is a set of instances of the form $\langle X_i, Y_i \rangle, i=1, \dots, N$.

MLE for Bayesian networks

- Our goal (in MLE) is to find the (set of) parameters θ that maximize the likelihood of the data.
- Due to the graph decomposition the likelihood function $L(\theta|D)$ decomposes into a set of two terms, one for each random variable X and Y .
- Each term is called the local likelihood function, which measures how it can be predicted given its parents.

MLE for Bayesian networks

- The ML estimates give,

$$\theta_{x=0} = \frac{M_{x=0}}{M_{x=0} + M_{x=1}} \quad \theta_{x=1} = \frac{M_{x=1}}{M_{x=0} + M_{x=1}}$$

$$\theta_{y=i|x=j} = \frac{M_{x=j,y=i}}{M_{x=j}}$$

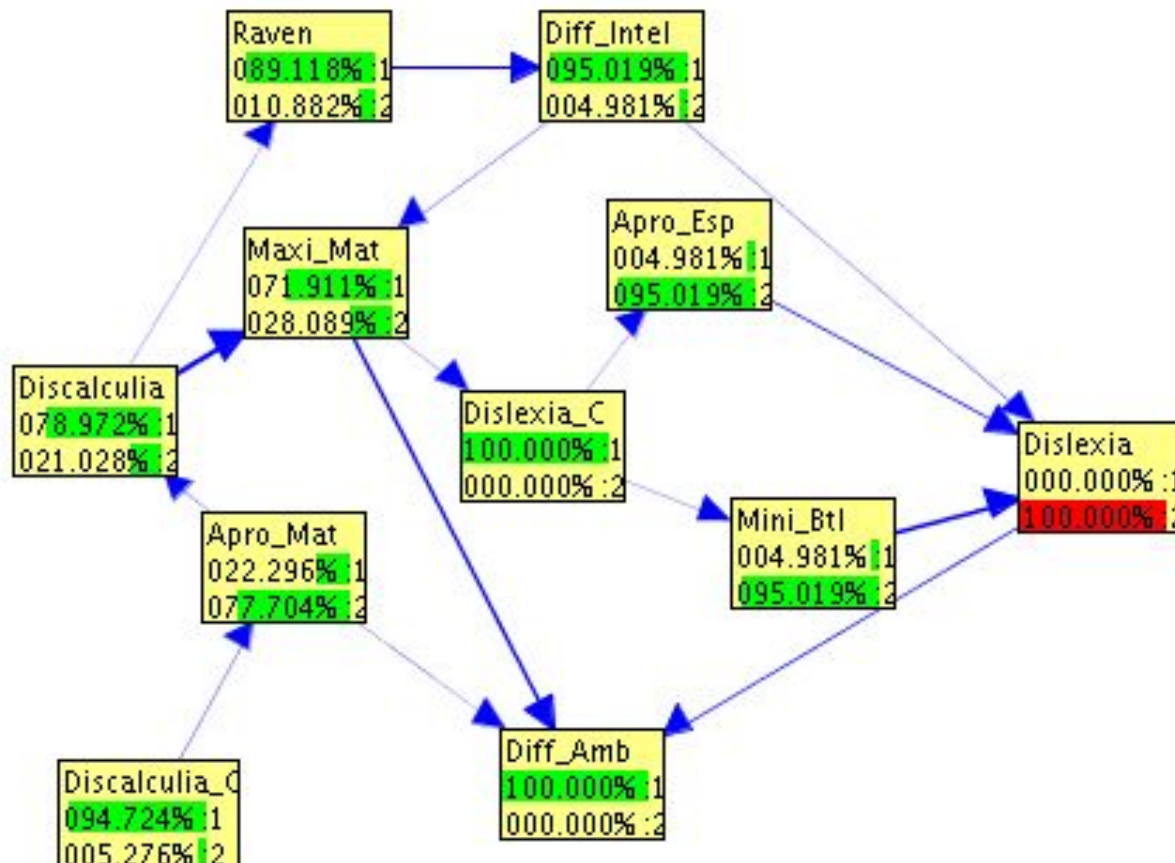
MLE for Bayesian networks

- These can then be combined to form the MLE solution for the whole network.
- For each random variable X with parents U , the MLE parameter is

$$\theta_{x|u} = M[x,u]/M[u]$$

where $M[u] = \sum_x M[x,u]$.

- For larger networks, the MLE parameter estimates are obtained by independently estimating the parameters for each CPD using the local likelihood function.



Working with Bayesian Networks

- Parameters learning: CDF estimation
- **Structure learning**
- Bayesian Inference

Structure learning/identification

- Structure learning refers to the estimation of the set of edges.
 - Domain expert who can arrange a suitable hierarchical model, or
 - It may be learned from data without any prior domain knowledge.

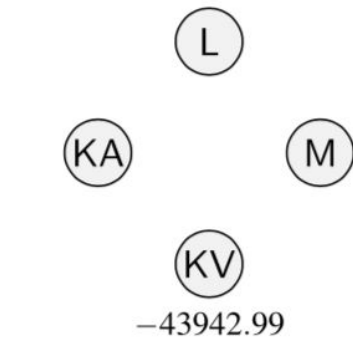
Bayesian Graphical Models

Structure learning (model selection)

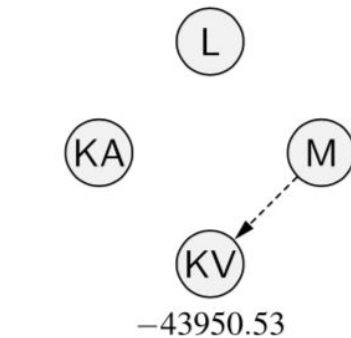
- Learning the optimal structure is NP-hard.
- Heuristic search through space of DAGs or node orderings.
- **Algorithms:** hill climbing, simulated annealing, GAs
- **Scoring function:** marginal likelihood, or an approximation like BIC/MDL or AIC, Hartley information gain (relational networks), Shannon information gain, χ^2 , K2 metric (probabilistic networks).

$$\begin{aligned} G^* &= \arg \max_G \log P(D|G)P(G) \\ &= \log \int_{\theta} P(D|G, \theta)P(\theta|G) \\ &\stackrel{BIC}{\approx} \log P(D|G, \theta^{ML}) - \lambda \dim(G) \end{aligned}$$

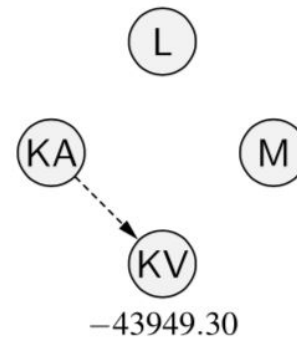
The structure learning landscape



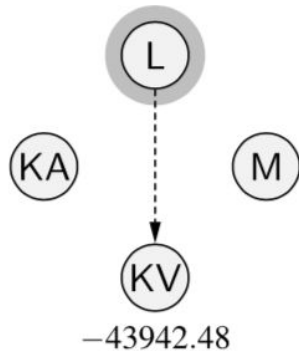
Step 1:
edgeless graph



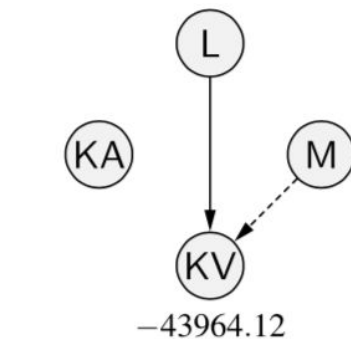
Step 2:
temporary insertion of M



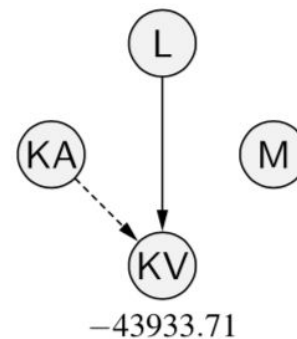
Step 3:
temporary insertion of KA



Step 4:
node L maximizes K2 value,
becomes first parent of KV.



Step 5:
temporary insertion of M.



Step 6:
node KA is added as second
parent of KV.

- **Constraint-based structure learning**
- **Score-based structure learning**
- **Precision matrix approaches**

Graphical lasso

Inverse of the covariance matrix (precision matrix).

Assumes that the joint distribution of attributes follows a Gaussian distribution.

$$(\mathcal{S}^{\text{opt}})^{-1} = \begin{bmatrix} 1.4500 & 0.0949 & 0.0480 & -0.0003 & -0.0155 & -0.0029 \\ 0.0949 & 1.4500 & 0.0036 & -0.0013 & -0.2360 & -0.0106 \\ 0.0480 & 0.0036 & 1.4500 & -0.0081 & -0.0035 & -0.0674 \\ -0.0003 & -0.0013 & -0.0081 & 1.4500 & 0.0077 & 0.1738 \\ -0.0155 & -0.2360 & -0.0035 & 0.0077 & 1.4500 & 0.0640 \\ -0.0029 & -0.0106 & -0.0674 & 0.1738 & 0.0640 & 1.4500 \end{bmatrix}$$



Working with Bayesian Networks

- Parameters learning: CDF estimation
- Structure learning
- **Bayesian Inference**

Evidence propagation

- To draw inferences when new information (evidence) becomes known.
- Given a graphical model (Markov or Bayes net) $G = (V, E)$ with underlying distribution p_V and an observed value a_0 of the attribute $A_0 \in V$.
- Propagating this evidence corresponds formally to the calculation of the following probability:

$$P(A=a|A_0=a_0)$$

$$\forall A \in V \setminus \{A_0\}$$

$$\forall a \in \text{dom}(A)$$

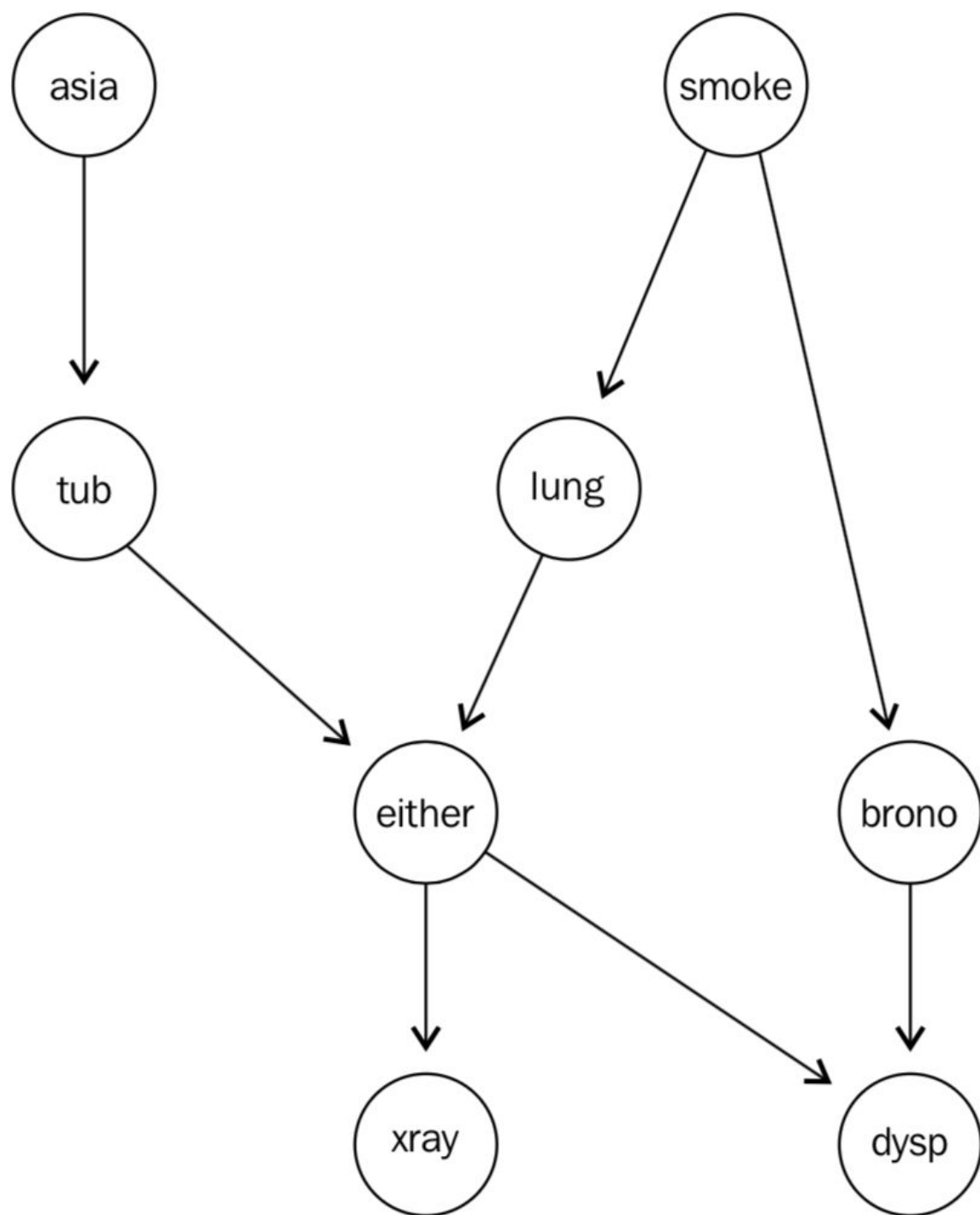
Evidence propagation

- Existing algorithms can be distinguished by:
 - Model type (Markov or Bayes network).
 - Graph topology (tree, polytree, general graph or completely independent of any graph structure).
 - Type of computation (exact or approximate).

Using the Variable Elimination algorithm

- To exemplify:
 - The Asia network
(<http://www.bnlearn.com/bnrepository/#asia>) is a toy Bayesian network used for patient diagnosis.
- The network tries to elicit the possibility of lung cancer or tuberculosis based on factors such as travel to Asia, smoking history, and X-ray results.

Further details on the Asia network can be found at
http://www.norsys.com/tutorials/netica/secA/tut_A1.htm



Using the Variable Elimination algorithm

- We shall now look at the Variable Elimination algorithm, which is an efficient method to calculate probabilities.
 - Avoiding duplicate operations.
 - Avoiding calculation of probabilities that are conditionally independent.

Using the Variable Elimination algorithm

- Representing factors:

$$\phi(\text{asia}) = P(\text{asia})$$

$$\phi(\text{smoke}) = P(\text{smoke})$$

$$\phi(\text{asia}, \text{tub}) = P(\text{tub}|\text{asia})$$

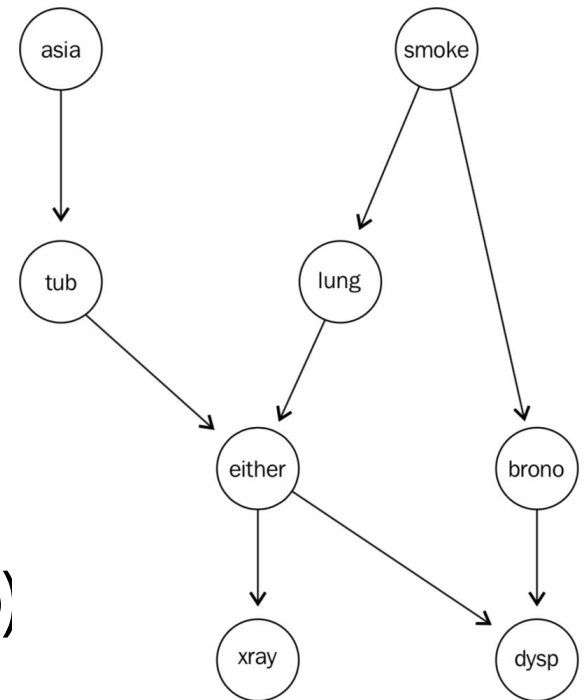
$$\phi(\text{smoke}, \text{lung}) = P(\text{lung}|\text{smoke})$$

$$\phi(\text{either}, \text{lung}, \text{tub}) = P(\text{either}|\text{lung}, \text{tub})$$

$$\phi(\text{bronc}, \text{smoke}) = P(\text{bronc}|\text{smoke})$$

$$\phi(\text{xray}, \text{either}) = P(\text{xray}|\text{either})$$

$$\phi(\text{dysp}, \text{bronc}, \text{either}) = P(\text{dysp}|\text{bronc}, \text{either})$$



Using the Variable Elimination algorithm

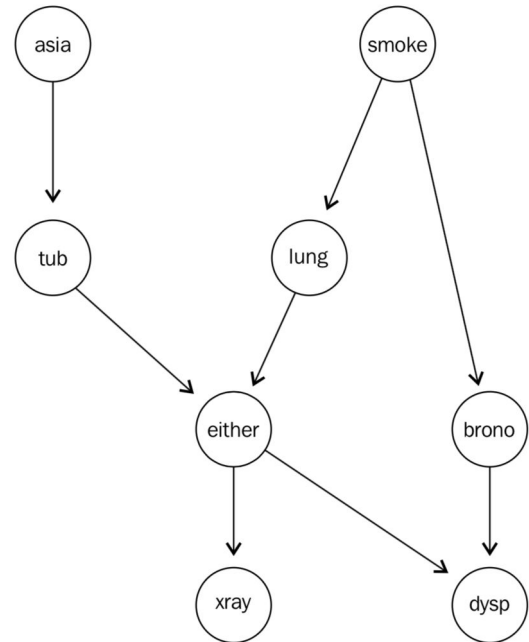
- Let us to infer the **$P(\text{bronc}|\text{xray})$** which corresponds to the factor $\phi(\text{xray}, \text{bronc})$.
- To create this factor, the **factor product operation** is applied.
- E.g., the set $\{\phi(\text{smoke}, \text{lung}), \phi(\text{smoke}, \text{bronc})\}$ has three variables in its combined scope: $\{\text{lung}, \text{smoke}, \text{bronc}\}$.
- Therefore, the product of factors will give us a $\phi(\text{smoke}, \text{lung}, \text{bronc})$ factor.

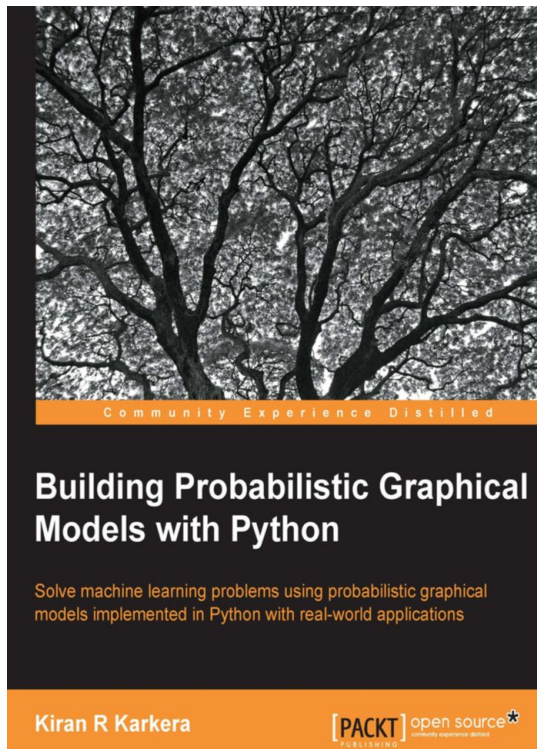
Using the Variable Elimination algorithm

- Thus, the idea of variable elimination is to eliminate variables one at a time.
- Each step consists of the following operations:
 - Multiplying factors.
 - Marginalizing a variable (which is in the scope of all multiplied factors).
 - Producing a new factor.

Step No	Factor product	Eliminated variable	New Factor created
1	$\phi(asia) \times \phi(tub, asia)$	asia	$\phi_1(tub)$
2	$\phi(smoke) \times \phi(smoke, lung) \times \phi(smoke, bronc)$	smoke	$\phi_2(lung, bronc)$
3	$\phi(either, lung, tub) \times \phi_1(tub),$	tub	$\phi_3(either, lung)$
4	$\phi_2(lung, bronc) \times \phi_3(either, lung),$	lung	$\phi_4(bronc, either).$
5	$\phi_4(bronc, either) \times \phi(dysp, bronc, either) \times \phi(xray, either),$	either	$\phi_5(dysp, bronc, xray)$
6	$\phi_5(dysp, bronc, xray),$	dysp	$\phi_6(xray, bronc)$

$$\varphi_6(xray, bronc) = P(bronc|xray)$$





Graphical Models

Representations for Learning,
Reasoning and Data Mining
Second Edition

Christian Borgelt
European Centre for Soft Computing, Spain

Matthias Steinbrecher & Rudolf Kruse
Otto-von-Guericke University Magdeburg, Germany

